

Preliminary Design Review

Project: Real-Time League of Legends Win Probability Overlay

Target Architecture: Fully Local / Personal Use (BYOK)

1. Executive Summary

This Preliminary Design Review (PDR) outlines the architecture and technical requirements for a real-time League of Legends win probability indicator. The system utilizes a neural network to evaluate both pre-game draft synergies and live in-game metrics. Designed strictly as a fully local, personal-use tool, the application leverages a "Bring Your Own Key" (BYOK) model to bypass the need for centralized server infrastructure while ensuring compliance with Riot Games' terms for individual developers.

2. System Architecture (Fully Local)

The application runs entirely on the host machine, handling data ingestion, model inference, and UI rendering concurrently without relying on external backend servers (other than Riot APIs).

- **API Authentication:** The user provides a personal Riot Developer API key upon initial startup, which is stored securely in a local configuration file.
- **Game State Detection:** The system continuously polls the local League of Legends Live Client Data API (127.0.0.1:2999).
- **Player Resolution:** Upon entering the loading screen, the application extracts the primary user's PUUID and queries the Riot Spectator API to resolve the identities and historical data of all 10 participants in the lobby.

3. Neural Network & Data Pipeline

3.1 Model Parameters

The prediction model relies on a composite feature set separated into two distinct game phases:

Phase	Primary Features	Data Source
Pre-Game	Champion mastery, historical win rates, team scaling, draft synergies.	Riot Match-v5 & Spectator APIs
In-Game	Gold differential, objective advantages (dragons/towers), game time, level gaps.	Live Client Data API

3.2 Data Ingestion & Training

Data Pipeline Protocol: Building the training dataset requires querying hundreds of thousands of historical matches. A custom Python API wrapper must be implemented with strict status code handling and exponential backoff to respect Riot's rate limits and prevent data loss during ingestion.

Raw JSON payloads from the API will be flattened and structured into a relational SQL database. This schema allows for efficient querying and normalization prior to model training in PyTorch. Once training is complete, the neural network weights will be exported to the **ONNX** format. Using `onnxruntime` locally ensures lightweight, high-speed inference without degrading the host machine's gaming performance.

3.3 Validation & Subsetting

Model accuracy heavily depends on the current meta and skill bracket. The training pipeline will isolate datasets by rank (e.g., separating Diamond+ from lower tiers) and current patch number. Validating the model's feature weights and prediction paths will be conducted using Tableau to visually confirm that the neural network accurately weights early-game gold leads against late-game scaling compositions.

4. Overlay Design (UI/UX)

The overlay prioritizes minimal visual intrusion. It consists of a single numerical display ranging from 0% to 100%.

4.1 Window Management

The UI will be constructed using Python's desktop frameworks (such as PyQt6 or CustomTkinter) with the following OS-level window flags:

- **Frameless:** Stripping native OS window borders (`FramelessWindowHint`).
- **Always on Top:** Ensuring the overlay remains visible above the fullscreen or borderless game client.

- **Click-Through:** Applying transparent window styles (e.g., `WS_EX_TRANSPARENT` on Windows) so mouse clicks pass directly through the text to the game underneath.

4.2 Draggable State Toggling

To allow user positioning, a global hotkey (e.g., `Ctrl+Shift+0`) will toggle the overlay's interactable state. When unlocked, the user can click and drag the percentage text to their preferred location (typically the top right corner). When locked, the click-through property is re-applied to prevent accidental interference during gameplay.